# Relatório – Pipeline CI/CD com IA Generativa (Germinaflix)

- **Repositório:** https://github.com/ArthurMachado2501/Germinaflix
- **Site publicado:** https://arthurmachado2501.github.io/Germinaflix/
- **Último run verde da pipeline (push na main):** https://github.com/ArthurMachado2501/Germinaflix/actions/runs/42606125771
- **Iterações:** 1 (checklist C1 a C12 passou, com a ressalva do C7 descrita em `ANALISE.md`)

## 1. Pipeline (`.github/workflows/ci-cd.yml`)

```
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  ci:
    name: ci
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Configurar Node
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Instalar dependências
        run: npm ci

      - name: Lint de HTML
        run: npm run lint

      - name: Conferir links locais
        run: npm run check:links

  deploy:
    name: deploy
    needs: ci
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
```

```
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    concurrency:
      group: pages
      cancel-in-progress: false
    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Configurar Pages
        uses: actions/configure-pages@v6

      - name: Montar o site (só HTML, CSS e imgs)
        run: |
          mkdir _site
          cp ./*.html ./*.css _site/
          cp -r imgs _site/imgs

      - name: Enviar artefato
        uses: actions/upload-pages-artifact@v5
        with:
          path: _site

      - name: Publicar no GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v5

      - name: Confirmar que o site está no ar
        env:
          URL: ${{ steps.deployment.outputs.page_url }}
        run: |
          for i in 1 2 3 4 5 6; do
            code=$(curl -s -o page.html -w '%{http_code}' "$URL" || true)
            echo "tentativa $i: HTTP $code"
            if [ "$code" = "200" ] && grep -q "Achados do Tutu" page.html; then
              echo "OK: $URL respondeu 200 com o texto esperado"
              exit 0
            fi
            sleep 10
          done
          echo "Falhou: $URL não respondeu 200 com 'Achados do Tutu'"
          exit 1
```

Arquivos de apoio: `package.json`, `package-lock.json`, `.htmlvalidate.json` e `scripts/check-links.mjs` (confere todo `href`/`src` local com a grafia exata do arquivo).

## 2. Prompt usado

Preciso realizar uma atividade de Pipeline CI/CD com IA Generativa O projeto é meu: https://github.com/ArthurMachado2501/Germinaflix É um site estático de catálogo de filmes, só com HTML, CSS e imagens. Hoje ele não tem nada

Pode clonar o repositório, criar branches, commitar, dar push, abrir e mergear PR e habilitar o GitHub Page, apenas no Germinaflix.

**Objetivo**

cada linha como um item de checklist que precisa de uma evidência concreta, e não apenas de uma afirmação sua:

- **C1.** A CI roda a cada `push` na `main`.
- **C2.** A CI roda também em `pull_request` para a `main`.
- **C3.** Faz o checkout do projeto.
- **C4.** Configura o ambiente necessário (por exemplo, Node com `actions/setup-node`).
- **C5.** Instala as dependências (`npm ci`, com lockfile commitado).
- **C6.** Executa uma etapa real de teste, validação ou verificação do projeto.
- **C7.** O CD roda automaticamente após um push na `main` e somente se a CI passou. Em pull request, não pode haver deploy.
- **C8.** O deploy usa uma estratégia adequada ao projeto. Para site estático, proponho GitHub Pages com `configure-pages`, `upload-pages-artifact` e `deploy-pages`, `permissions` mínimas, `environment: github-pages` e `concurrency`.
- **C9.** A nova versão fica acessível numa URL pública, e a própria pipeline confirma isso com um `curl` na URL (HTTP 200 e um texto esperado, como "Achados do Tutu").
- **C10.** O arquivo da pipeline está em `.github/workflows/`.
- **C11.** O registro do processo está completo (explico abaixo).
- **C12.** Existe um relatório final em um único PDF.

**Como você vai trabalhar: um workflow em rodadas**

Você vai executar os 7 passos abaixo. No fim do passo 6, se algum item do checklist falhou, você volta ao passo 2 e refaz só o que falhou. Isso pode acontecer no máximo 4 vezes. Cada volta completa pelos passos é uma **iteração**.

**Regra do registro:** durante os passos 1 a 5 não escreva diário nem anotações soltas, só trabalhe. O registro oficial acontece **uma única vez por iteração, no passo 6**, quando você já sabe o resultado. Assim ele sai fiel e sem enrolação.

**Passo 1 — Conhecer o terreno (só na primeira iteração)**

1. Clone o repositório (ou use o clone atual) e leia todos os HTML e CSS.
2. Descubra qual é a branch padrão e qual será a URL do Pages (provavelmente `https://arthurmachado2501.github.io/Germinaflix/`).
3. Fique atento a dois riscos que costumam derrubar este tipo de projeto. Primeiro, o site fica num subcaminho (`/Germinaflix/`), então links começando com `/` quebram. Segundo, o runner é Linux e eu desenvolvo no Windows, então `imgs/Alee.jpg` e `imgs/alee.jpg` são arquivos diferentes lá e dão 404.

**Passo 2 — Planejar**

1. Em `docs/lab-cicd/PLANO.md`, descreva a estratégia de CI: qual ferramenta de validação de HTML usar e por quê (por exemplo `html-validate` ou `htmlhint`), mais um pequeno script em Node que confira se todo `href` e `src` local aponta para um arquivo que existe, respeitando maiúsculas e minúsculas.
2. Descreva a estratégia de CD e justifique por que GitHub Pages é adequado a um site estático.
3. Liste, em ordem, o que será feito nesta iteração e como saberemos que cada item do checklist está pronto.
4. A partir da segunda iteração, planeje apenas os itens que falharam.

**Passo 3 — Construir**

1. Crie uma branch `feat/ci-cd` e trabalhe em commits pequenos.
2. Adicione `package.json` e o lockfile, o script de checagem de links locais e o lint.
3. Crie um único workflow, `.github/workflows/ci-cd.yml`, com um job `ci` e um job `deploy`. O `deploy` deve ter `needs: ci` e `if: github.event_name == 'push' && github.ref == 'refs/heads/main'`. Confira as versões atuais das actions antes de fixá-las.
4. Habilite o Pages com origem GitHub Actions (`gh api -X POST repos/ArthurMachado2501/Germinaflix/pages -f build_type=workflow`; se já existir, use PUT).
5. Publique só o que o site precisa (HTML, CSS e `imgs/`), sem `node_modules` nem `docs/`.

**Passo 4 — Testar localmente**

Rode na minha máquina tudo o que der: instalar dependências, lint, script de links. Corrija o que quebrar antes de subir qualquer coisa.

**Passo 5 — Testar no GitHub de verdade**

1. Abra um PR da branch para a `main` e acompanhe com `gh run watch`. Confirme que a CI roda e que o job `deploy` fica *skipped*.
2. Se o lint ou o script encontrarem problemas reais no site, deixe a CI falhar uma vez e depois corrija. Isso me ajuda a mostrar que a pipeline realmente pega erros. Nunca provoque nem invente uma falha artificial.
3. Faça o merge na `main` e confirme que CI e, depois, deploy rodam em sequência e passam.
4. Abra a URL publicada (`curl -sI` e `curl -s | grep`) e, se houver navegador disponível, tire um screenshot.
5. Prove que o gate do C7 funciona: com um PR descartável que tenha um erro real e que será fechado sem merge, mostre que o deploy não executa quando a CI falha. Nunca quebre a `main` para isso.

**Passo 6 — Avaliar e registrar os achados (um registro por iteração)**

Agora sim, escreva. Seja cético: item sem evidência concreta conta como falhou.

1. Em `docs/lab-cicd/ANALISE.md`, marque C1 a C12 como PASSOU ou FALHOU, cada um com sua evidência (comando executado, link do run via `gh run view --json url` ou arquivo em `docs/lab-cicd/evidencias/`).
2. Em `docs/lab-cicd/PROCESSO.md`, acrescente uma seção "Iteração N" com: o que foi tentado, o que deu certo, os problemas encontrados, as alterações feitas e quem fez cada uma (você ou eu, manualmente).

Registre apenas o que de fato aconteceu.
3. Salve as saídas relevantes de comandos e de runs em `docs/lab-cicd/evidencias/`.

**Passo 7 — Decidir**

- Se tudo passou: vá para a entrega.
- Se algo falhou e a iteração atual for menor que 4: volte ao Passo 2 apenas com os itens que falharam.
- Se chegou à iteração 4 com falhas: pare e me diga exatamente o que falta e por quê, sem maquiar o resultado.

**Entrega**

Gere `docs/lab-cicd/RELATORIO.md` com:

- o link do repositório;
- o YAML completo da pipeline;
- este prompt e o resumo de cada iteração (vindo do `PROCESSO.md`);
- as respostas às 5 perguntas do laboratório: (1) quantos prompts foram necessários até a versão final, respondendo "1 prompt meu + N iterações autônomas" com os números reais; (2) principais problemas; (3) alterações necessárias; (4) alterações feitas manualmente; (5) como verificamos que a pipeline funciona;
- a justificativa da escolha do GitHub Pages e uma breve análise do processo;
- as evidências.

Converta para um único PDF em `docs/lab-cicd/RELATORIO.pdf` pela opção mais simples que existir (pandoc, ou Edge/Chrome headless imprimindo um HTML gerado). Se nada funcionar, deixe o `.md` pronto e me diga como converter. Não envie o formulário, isso é comigo.

**Ao terminar**

Me responda em no máximo 10 linhas, em linguagem simples: a URL publicada, o link do último run verde, quantas iterações foram feitas, o resultado de cada item do checklist e o que ainda preciso fazer manualmente (revisar o relatório, anexar screenshots que você não conseguiu tirar e enviar o PDF no formulário).

## 3. Resumo das iterações (de `PROCESSO.md`)

### Iteração 1

**Quem fez:** Claude executou tudo (git, `gh`, arquivos) com a conta GitHub logada do Arthur, a partir de um único prompt. O Arthur não fez nenhuma alteração manual no código; a única outra mensagem dele foi uma autorização complementar ("se precisar, controle a minha tela e edite no meu github"), que não precisou ser usada.

**O que foi tentado**

1. Clonei o repositório, li os 4 HTML e os 4 CSS e conferi a branch padrão (`main`) e a URL do Pages. O Pages já estava ativo, com origem "branch main" (`build_type: legacy`).

2. Escrevi o `PLANO.md` (lint com `html-validate`, script de links, deploy com GitHub Pages).
3. Criei a branch `feat/ci-cd` e fiz 4 commits pequenos: `package.json` + lockfile + `.htmlvalidate.json`; `scripts/check-links.mjs`; `.github/workflows/ci-cd.yml`; plano e saída dos testes locais.
4. Troquei o Pages para origem GitHub Actions: `gh api -X PUT .../pages -f build_type=workflow` (PUT porque o Pages já existia).
5. Abri o PR #1, deixei a CI rodar, corrigi o site, fiz o merge e verifiquei o deploy.
6. Provei o gate com o PR descartável #2 (fechado sem merge, branch apagada).

**O que deu certo**

- Versões das actions conferidas pela API de releases antes de fixar (checkout v7.0.1, setup-node v7.1.0, configure-pages v6.0.0, upload-pages-artifact v5.0.0, deploy-pages v5.0.1).
- Nos testes locais, o script de links já apontou o `href=""` do filme 7 e o `html-validate` apontou 15 problemas, tudo antes de subir.
- PR #1: a CI falhou no primeiro run (lint, 15 erros reais) e o `deploy` ficou *skipped*. Depois da correção o run passou, ainda com `deploy` *skipped*.
- Merge na `main`: `ci` e `deploy` em sequência, verdes; `curl` na pipeline com HTTP 200 e "Achados do Tutu"; `curl` externo e screenshot confirmaram o site.
- PR descartável #2: CI vermelha por `Alee.jpg` (o arquivo real é `alee.jpg`) e `deploy` *skipped*.

**Problemas encontrados**

- `npm run lint` não roda localmente: o caminho da pasta contém `&` ("Instituto J&F") e o atalho do npm quebra (`Cannot find module 'C:\Users\arthurmachado-ieg\html-validate\bin\html-validate.mjs'`). Contorno só na máquina local: `node node_modules/html-validate/bin/html-validate.mjs "*.html"`. No runner Linux o `npm run lint` funciona normalmente.
- Erros reais no site:
  - `<meta http-equiv="refresh" content="5">` no `index.html`: a página recarregava sozinha a cada 5 segundos;
  - `href=""` no pôster do filme 7 (pego pelo script de links);
  - 10 elementos `<nav>` usados como caixa de pôster, sem nome acessível (`unique-landmark`);
  - link do Instagram com imagem sem `alt` (`wcag/h30`);
  - espaços no fim de linha em `index.html` (2) e `resenha_pecadores.html` (1).
- O Git avisa que converterá LF em CRLF nos arquivos novos (configuração local do Windows); não afetou a pipeline.

**Alterações feitas (todas por Claude)**

- Pipeline e tooling: `package.json`, `package-lock.json`, `.htmlvalidate.json`, `.gitignore`, `scripts/check-links.mjs`, `.github/workflows/ci-cd.yml`.
- Site (commit `fix: corrige problemas de HTML apontados pelo lint`): removido o meta refresh; `<nav>` trocado por `<div>` (o CSS não usava `nav`); removido o link vazio do filme 7 (ficou só a imagem, sem inventar um trailer); `alt="Instagram"` no ícone; espaços finais removidos.
- Configuração do GitHub: Pages com origem Actions.
- Documentos: `PLANO.md`, `ANALISE.md`, `PROCESSO.md`, `RELATORIO.md`, `RELATORIO.pdf`, `evidencias/`.

**Alterações manuais do Arthur:** nenhuma até aqui.

**Resultado da iteração:** C1 a C12 PASSOU (ver `ANALISE.md`, com a ressalva do C7). Sem segunda iteração.

## 4. Respostas às 5 perguntas do laboratório

**1. Quantos prompts foram necessários até a versão final?** 1 prompt meu + 1 iteração autônoma. Depois do prompt principal enviei só uma mensagem curta de autorização ("se precisar, controle a minha tela e edite no meu github"), que não precisou ser usada. A pipeline ficou pronta e verde na primeira iteração; o único "retrabalho" foi corrigir o site depois que a CI falhou de propósito no PR #1, o que o próprio roteiro previa (passo 5.2).

**2. Principais problemas**

- O lint achou erros reais no site: `meta refresh` de 5 segundos recarregando a página sozinha, `href=""` num pôster, 10 `<nav>` sem nome acessível, link do Instagram sem `alt` e espaços no fim de linha (15 erros no lint, mais o link vazio apontado pelo script).
- `npm run lint` não roda na minha máquina porque o caminho da pasta tem `&` ("Instituto J&F") e o atalho do npm quebra. Só afeta o ambiente local; contornei chamando o Node direto.
- O Pages já estava ligado com origem "branch main"; foi preciso trocar para "GitHub Actions" com `PUT` em vez de `POST`.
- Os dois riscos que eu temia (links com `/` no subcaminho e maiúsculas/minúsculas) não existiam no site atual, mas o script de links agora impede que apareçam.

**3. Alterações necessárias**

- Pipeline: `ci-cd.yml`, `package.json`, `package-lock.json`, `.htmlvalidate.json`, `scripts/check-links.mjs`, `.gitignore`.
- Site: remover o `meta refresh`, trocar `<nav>` por `<div>`, remover o link vazio, colocar `alt` no ícone do Instagram e tirar os espaços no fim das linhas.
- GitHub: trocar a origem do Pages para GitHub Actions.

**4. Alterações feitas manualmente** Nenhuma. Todas as alterações foram feitas pela IA (Claude) por meio de `git` e `gh`, usando minha conta já autenticada. Minhas ações manuais ficam para o fim: revisar o relatório e enviar o PDF no formulário.

**5. Como verificamos que a pipeline funciona**

- Localmente: `npm ci`, lint e script de links antes de subir qualquer coisa.
- No GitHub, com 4 runs reais: PR com CI vermelha (lint pegou erros reais, `deploy` skipped); PR corrigido com CI verde (`deploy` skipped); merge na `main` com `ci` e depois `deploy` verdes; PR descartável #2 com erro de maiúsculas (`Alee.jpg`), CI vermelha e `deploy` skipped, fechado sem merge.
- A própria pipeline faz `curl` na URL pública e exige HTTP 200 com "Achados do Tutu" (passou na primeira tentativa).
- Conferência de fora da pipeline: `curl -sI`, `curl -s | grep`, ausência do `meta refresh` na versão publicada (prova que é a versão nova), `docs/` com 404 e screenshot do site.
- Limite do teste: o caso "push na `main` com CI vermelha" não foi executado, porque exigiria quebrar a `main`. Nesse caso confiamos no `needs: ci`.

## 5. Por que GitHub Pages

O Germinaflix é 100% estático (HTML, CSS e imagens), então não precisa de servidor, banco nem container. O GitHub Pages é gratuito, fica no mesmo lugar do código, entrega HTTPS e CDN, e o deploy usa o `GITHUB_TOKEN`/OIDC do próprio workflow, sem segredos para guardar. A URL é a mesma que o projeto já usava. Netlify, Vercel ou Cloud Run exigiriam outra conta, tokens e possível custo sem nenhum ganho para este caso.

## 6. Análise do processo

- Funcionou bem deixar a pipeline subir com o site ainda sem correção: a falha do PR #1 mostrou que o lint pega erros de verdade e que o `deploy` não roda em PR.
- Rodar as verificações localmente antes de subir evitou surpresas; a diferença entre local e runner foi só o bug do `&` no caminho.
- O script de links compara o nome exato do arquivo no disco, então o erro `Alee.jpg` vs `alee.jpg`, que no Windows passaria despercebido, foi pego (PR #2).
- Pontos fracos: não há teste automatizado de aparência ou de navegação (só lint e links), e o gate `needs: ci` na `main` foi validado pela semântica do Actions, não por uma falha real. O `ubuntu-latest` mostra um aviso de migração para o Ubuntu 26; vale acompanhar.

## 7. Checklist e evidências

Repositório: https://github.com/ArthurMachado2501/Germinaflix · URL publicada: https://arthurmachado2501.github.io/Germinaflix/

Runs usados como prova (workflow `CI/CD`):

| Run | Evento | Branch | Resultado | Link |
|---|---|---|---|---|
| 42605936433 | pull_request | feat/ci-cd | **falhou** (lint achou erros reais); `deploy` *skipped* | https://github.com/ArthurMachado2501/Germinaflix/actions/runs/42605936433 |
| 42606045502 | pull_request | feat/ci-cd | passou; `deploy` *skipped* | https://github.com/ArthurMachado2501/Germinaflix/actions/runs/42606045502 |
| 42606125771 | push | main | `ci` e `deploy` passaram | https://github.com/ArthurMachado2501/Germinaflix/actions/runs/42606125771 |
| 42606298085 | pull_request | test/gate-ci-falha (PR #2, fechado sem merge) | **falhou** (link com maiúscula errada); `deploy` *skipped* | https://github.com/ArthurMachado2501/Germinaflix/actions/runs/42606298085 |

| Item | Resultado | Evidência |
|---|---|---|
| C1. CI roda a cada push na `main` | PASS OU | `on.push.branches: [main]` em `.github/workflows/ci-cd.yml`; run 42606125771 tem `event: push` na `main` (`evidencias/06-main-run.json`). |
| C2. CI roda em `pull_request` para a `main` | PASS OU | `on.pull_request.branches: [main]`; runs 42605936433, 42606045502 e 42606298085 têm `event: pull_request` (`evidencias/03`, `05`, `10`). |
| C3. Faz o checkout | PASS OU | Passo `Checkout` (`actions/checkout@v7`) com ✓ em `evidencias/06-main-run-watch.txt`. |

| Item | Resultado | Evidência |
|---|---|---|
| C4. Configura o ambiente | PASSOU | Passo `Configurar Node` (`actions/setup-node@v7`, Node 24, cache npm), ✓ no mesmo arquivo. |
| C5. Instala dependências com `npm ci` e lockfile commitado | PASSOU | `git ls-files package-lock.json` lista o arquivo; passo `Instalar dependências` (`npm ci`) ✓ no run 42606125771; teste local em `evidencias/01-local-iteracao1.txt`. |
| C6. Etapa real de verificação | PASSOU | `html-validate` (lint) e `scripts/check-links.mjs`. Prova de que são reais: o run 42605936433 falhou com 15 erros de lint no site (`evidencias/03-pr-run-falha.txt`) e o run 42606298085 falhou por `Alee.jpg` (`evidencias/10-gate-pr-descartavel.txt`). |
| C7. CD só após push na `main` e só se a CI passou; sem deploy em PR | PASSOU (com ressalva) | `deploy` tem `needs: ci` e `if: github.event_name == 'push' && github.ref == 'refs/heads/main'`. Observado: `deploy` *skipped* nos 3 runs de PR (inclusive o que teve CI vermelha, `evidencias/10`) e executado depois do `ci` no push da `main` (`evidencias/06`). **Ressalva:** o caso "push na `main` com CI vermelha" não foi testado de fato, porque exigiria quebrar a `main`. Em PR o `if` já pula o `deploy`, então o run 42606298085 prova o comportamento em PR; a garantia do `needs: ci` na `main` vem da semântica do GitHub Actions, não de um teste. |
| C8. Estratégia adequada ao projeto | PASSOU | `configure-pages@v6`, `upload-pages-artifact@v5`, `deploy-pages@v5`; `permissions` no topo `contents: read` e, no `deploy`, `pages: write` + `id-token: write`; `environment: github-pages`; `concurrency: pages`. `gh api repos/.../pages` retorna `build_type: workflow` (`evidencias/02-pages-config.txt`). Artefato com só HTML, CSS e `imgs/`; `docs/` retorna 404 no site (`evidencias/08-curl-externo.txt`). |
| C9. Nova versão em URL pública, confirmada por `curl` na pipeline | PASSOU | Passo `Confirmar que o site está no ar`: `tentativa 1: HTTP 200` e `OK: ... respondeu 200 com o texto esperado` (`evidencias/07-curl-na-pipeline.txt`). Fora da pipeline: HTTP 200, "Achados do Tutu" nas linhas 8 e 19 e **0** ocorrências de `http-equiv="refresh"`, ou seja, é a versão nova (`evidencias/08-curl-externo.txt`). Screenshot: `evidencias/09-screenshot-site-publicado.png`. |
| C10. Pipeline em `.github/workflows/` | PASSOU | `.github/workflows/ci-cd.yml` (`git ls-files .github/workflows`). |
| C11. Registro do processo completo | PASSOU | `PLANO.md`, `PROCESSO.md` (seção "Iteração 1"), este `ANALISE.md` e `evidencias/` (01 a 10). |
| C12. Relatório final em um único PDF | PASSOU | `docs/lab-cicd/RELATORIO.pdf`, gerado do `RELATORIO.md` (ver `evidencias/11-pdf.txt`). |

## Decisão (Passo 7)

Nenhum item falhou na iteração 1. Não há segunda iteração.

## Evidência A – CI falhando em PR por erros reais (run 42605936433)


```
{"conclusion":"failure","event":"pull_request","jobs":[{"conclusion":"failure","name":"ci"},
{"conclusion":"skipped","name":"deploy"}],"url":"https://github.com/ArthurMachado2501/Germinaflix/actions/r
```

```
uns/42605936433"}
ci      Lint de HTML    2026-11-09T11:03:55.4818128Z ##[group]Run npm run lint
ci      Lint de HTML    > html-validate "*.html"
ci      Lint de HTML       8:41  error  Meta refresh must be instant (0 second delay)
meta-refresh
ci      Lint de HTML      37:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML      38:33  error  Trailing whitespace
no-trailing-whitespace
ci      Lint de HTML      50:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML      60:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML      70:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML      80:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML     101:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML     114:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML     124:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML     134:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML     142:31  error  Trailing whitespace
no-trailing-whitespace
ci      Lint de HTML     144:18  error  Landmarks must have a non-empty and unique accessible name (aria-
label or aria-labelledby)  unique-landmark
ci      Lint de HTML     157:10  error  Anchor link must have a text describing its purpose
wcag/h30
ci      Lint de HTML     33:56  error  Trailing whitespace  no-trailing-whitespace
ci      Lint de HTML   ✖ 15 problems (15 errors, 0 warnings)
ci      Lint de HTML     3 errors and 0 warnings potentially fixable with the `--fix` option.
ci      Lint de HTML     https://html-validate.org/rules/meta-refresh.html
ci      Lint de HTML     https://html-validate.org/rules/unique-landmark.html
ci      Lint de HTML     https://html-validate.org/rules/no-trailing-whitespace.html
ci      Lint de HTML     https://html-validate.org/rules/wcag/h30.html
```

## Evidência B – PR corrigido: CI verde, deploy skipped (run 42606045502)

```
{"conclusion":"success","event":"pull_request","jobs":[{"conclusion":"success","name":"ci"},
{"conclusion":"skipped","name":"deploy"}],"url":"https://github.com/ArthurMachado2501/Germinaflix/actions/r
uns/42606045502"}

✓ feat/ci-cd CI/CD ArthurMachado2501/Germinaflix#1 · 42606045502
Triggered via pull_request less than a minute ago

JOBS
✓ ci in 8s (ID 129122194850)
- deploy in 0s (ID 129122273543)

ANNOTATIONS
```

## Evidência C – merge na main: ci e deploy em sequência (run 42606125771)

{"conclusion":"success","event":"push","jobs":[{"conclusion":"success","name":"ci"},
{"conclusion":"success","name":"deploy"}],"url":"https://github.com/ArthurMachado2501/Germinaflix/actions/runs/42606125771"}

```
deploy  Confirmar que o site está no ar       ^[[36;1m  echo "tentativa $i: HTTP $code"^[[0m
deploy  Confirmar que o site está no ar       ^[[36;1m    echo "OK: $URL respondeu 200 com o texto esperado"^[[0m
deploy  Confirmar que o site está no ar       tentativa 1: HTTP 200
deploy  Confirmar que o site está no ar       OK: https://arthurmachado2501.github.io/Germinaflix/ respondeu 200 com o texto esperado
```

## Evidência D – verificação externa da URL publicada

```
$ curl -sI https://arthurmachado2501.github.io/Germinaflix/
HTTP/1.1 200 OK
Connection: keep-alive
Content-Length: 6600
Server: GitHub.com
Content-Type: text/html; charset=utf-8
Last-Modified: Mon, 09 Nov 2026 11:05:32 GMT
Access-Control-Allow-Origin: *
Strict-Transport-Security: max-age=31556952

$ curl -s https://arthurmachado2501.github.io/Germinaflix/ | grep -n 'Achados do Tutu'
8:    <title>Achados do Tutu</title>
19:        <h1>Achados do Tutu</h1>

meta refresh na versao publicada (esperado: 0):
0

sobre_o_desenvolvedor.html -> 200
resenha_pecadores.html -> 200
style.css -> 200
imgs/jef.png -> 200
imgs/alee.jpg -> 200
docs/ publicado? -> 404
```




## Evidência E – gate: PR descartável com erro real (run 42606298085)

```
{"conclusion":"failure","event":"pull_request","jobs":[{"conclusion":"failure","name":"ci"},
{"conclusion":"skipped","name":"deploy"}],"url":"https://github.com/ArthurMachado2501/Germinaflix/actions/r
uns/42606298085"}
ci      Conferir links locais  ✗ 1 problema(s) de link local:
ci      Conferir links locais    sobre_o_desenvolvedor.html:48 src="./imgs/Alee.jpg" -> arquivo não existe
(ou grafia diferente)
```

## Evidência F – Pages com origem GitHub Actions

```
{"build_type":"workflow","html_url":"https://arthurmachado2501.github.io/Germinaflix/","status":"built"}
```

## Evidência G – testes locais (iteração 1)

```
$ npm ci

added 13 packages, and audited 14 packages in 2s

3 packages are looking for funding
  run `npm fund` for details
```

```
found 0 vulnerabilities

$ npm run lint

> germinaflix@1.0.0 lint
> html-validate "*.html"

O sistema n◆o pode encontrar o caminho especificado.
node:internal/modules/cjs/loader:1520
  throw err;
  ^

Error: Cannot find module 'C:\Users\arthurmachado-ieg\html-validate\bin\html-validate.mjs'
    at Module._resolveFilename (node:internal/modules/cjs/loader:1517:15)
    at wrapResolveFilename (node:internal/modules/cjs/loader:1071:27)
    at defaultResolveImplForCJSLoading (node:internal/modules/cjs/loader:1095:10)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1122:12)
    at Module._load (node:internal/modules/cjs/loader:1294:5)
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v24.19.0
exit=1

$ npm run check:links

> germinaflix@1.0.0 check:links
> node scripts/check-links.mjs

✗ 1 problema(s) de link local:
  index.html:114 href="" -> valor vazio
exit=1
$ node node_modules/html-validate/bin/html-validate.mjs "*.html"   (contorno do bug do '&' no caminho)

C:\Users\arthurmachado-ieg\OneDrive - Instituto J&F\Área de Trabalho\SOLIER\Germinaflix\index.html
    8:41  error  Meta refresh must be instant (0 second delay)
meta-refresh
   37:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
   38:33  error  Trailing whitespace
no-trailing-whitespace
   50:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
   60:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
   70:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
   80:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
  101:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
  114:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
```

```
unique-landmark
  124:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
  134:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
  142:31  error  Trailing whitespace
no-trailing-whitespace
  144:18  error  Landmarks must have a non-empty and unique accessible name (aria-label or aria-labelledby)
unique-landmark
  157:10  error  Anchor link must have a text describing its purpose
wcag/h30
```